

PROJECT SPECS/EXPLANATION

An 8×8 parameterizable INT8 systolic-array matrix accelerator on FPGA, with output-stationary accumulation, BRAM-backed tile buffers, software-controlled tiling, and Python-based verification/benchmarking.

ARM software performs matrix tiling and accelerator scheduling, while a custom FPGA systolic array performs the compute.

INPUT: INT8 Matrix (8 bit signed integer from -128 to 127)

Matrix A: 8 by K ($K = 64 = \text{length of Dot Product}$)

Matrix B: K by 8 ($K = 64$)

Matrix C: 8 by 8

Parameterizable: Design can be changed with just parameters, instead of rewriting the whole module

PE (Processing Element): A PE is just a tiny hardware block that does one multiply-accumulate operation (MAC) each cycle: $\text{Acc} < \text{Acc} + A \times B$. Contains one multiplier, one adder, one accumulation reg, and data registers to pass data around.

PE takes in A_{in} and B_{in} , and pumps out A_{out} and B_{out} . 8x8 systolic array will have 64 PEs. Each PE builds one value of the 8x8 result

Systolic-array: a grid of small PEs where data is passed from one unit to the next every clk cycle, instead of every unit repeatedly going back to memory.

Benefits of SA: Many PEs work at the same time, so we get parallel computation. Data gets reused, instead of constantly extracting new data from memory. One A value can be reused by the PEs down a row and same with B. This is important because often the bottleneck is not the multiplier speed, it is getting enough data from memory into the compute units fast enough.

Output-stationary accumulation: Each PE keeps one output in place while its being calculated.

Cycle 1: $\text{acc} += A_{00} \times B_{00}$

Cycle 2: $\text{acc} += A_{01} \times B_{10}$

Cycle 3: $\text{acc} += A_{02} \times B_{20}$

Acc is being tracked in the PE perma until the very end when its the real value.

BRAM = Block RAM. It's dedicated on-chip memory inside the FPGA.

Tile buffer = a small memory that stores one matrix "tile" or chunk.

BRAM-backed tile buffers means using the FPGA's built-in memory blocks to temporarily store the chunk of matrix data thats is going to be computed.

One tile: 8 x 64 INT8, these get loaded into BRAM before computation starts. Systolic Array needs memory data every cycle, BRAM is fast, local, and better than fetching memory from a PC.

Software-controlled tiling: Python/C++ side breaks a big matrix into smaller chunks that fit the hardware accelerator. Since the hardware works on 64x8, and the real matrix could potential be larger, software breaks down the larger matrix into data that the hardware can handle. Software also handles matrix combination.

Python-based verification/benchmarking: fancy way of saying using python to check hardware answer XD (Golden reference model). Also can be used to check the speed.

LLM can be used as an extension if time permits.

C++ and python are pretty much interchangeable.

PROJECT PLANNER:

Stage 1: Prep

Date: August 11 to 14

Goals: Lock in project specs, architecture discussion, research key elements of the accelerator specs.

Collab: One meeting (virtual/online) to finalize.

Leader: N/A

Stage 2: Software Prep

Date: August 14 - 15

Goals: Create Python/C++ code that can verify the hardware later (Golden Model). Create Python/C++ code that handles matrix tiling/packing from larger matrix to 8x64 and back. INT8/32 matrix generation and calculation is also needed. Essentially making a software test bench. Include run-time, edge cases, and comparison functions if needed.

Collab: One wrap-up meeting if needed.

Leader:

- Golden model = ground truth (results of matrix multiplication)
- use c++ to manually verify matrix multiplication
- make sure to use 8 bit signed int for the PEs
- do the tiling system (for large matrix inputs)

Stage 3: Build unit test PE

Date: August 15 - 18

Goals: Create one PE with A_reg, B_reg, multiplier, INT32 accumulator, A->right, B->down (for moving through the systolic array). PE should be created using verilog and tested via modelsim. PE will carry out $arr < arr + AXB$. A modelsim test bench should also be created with edge cases. Test for positive/negative numbers accumulation, reset, overflow assumptions, valid signals.

Collab: One wrap-up meeting if needed. Both Members should operate and understand the PE building, as well as verify the modelsim waveform.

Leader:

Stage 4: Build unit test 2xk systolic array

Date: August 18 - 22

Goals: Build a 2 x k systolic array and manually verify the data movement to ensure solid logic. Additionally, we need to solve input timing issues to ensure that data arrives at the correct PE on clk cycle. Finally we need to check via modelsim that parameterized 2x2 - 8x8 arrays pass simulation to ensure that logic is solid.

Collab: One wrap-up meeting. Both members should contribute to the PE building and modelsim waveform.

Leader:

Stage 5: Add memory/controller

Date: August 22 - 26

Goals: Build the actual accelerator around the systolic array while incorporating BRAM. Build two BRAM tile buffers for input and one for output. Build a controller/FSM, Build signals and controllers for computational flow. Hardware should produce a result at this point.

Collab: One member should focus on BRAM, the other should focus on FSM. However, Both members should be involved in both parts, and fully understand the logic behind it.

Leader: N/A

Stage 6: Hardware verification

Date: August 26 - Sept 1

Goals: Build the modelsim testbenches for the entire accelerator and verify memory/data flow. Compare hardware output on modelsim to python/c++ output. Test for edge cases, software tiling and program flow. Time the accelerator.

Collab: One in person meeting since this is the last portion of virtual testing.

Leader:

Stage 7: Grace Period

Date: Sept 1 - 3

Goals: Complete any unfinished work from the previous stages and be fully ready for FPGA labs and testing.

Stage 8: FPGA testing and synthesis

Date: Sept 3-7

Stage 9: Benchmarking and optimization

Date: Sept 7 - 10

Stage 10: Extension + Polish + Finalization

Date: Sept 10-15